

IT-Engineering Collaboration Dashboard

Date: 2026-03-09

Contents

1	Introduction	2
2	System Architecture	2
3	Key Features	2
3.1	Team Dashboard	2
3.2	Task Management	3
3.3	Calendar and Event Scheduling	3
3.4	Equipment Booking	3
3.5	Personnel Tracking	3
3.6	Notifications	3
3.7	Additional Features	3
4	Implementation Details	4
4.1	Authentication	4
4.2	API Routes	4
4.3	Database	4
4.4	Frontend Components	4
4.5	Background Tasks	4
5	Conclusion	5

1 Introduction

IT and Engineering teams in organisations commonly rely on a fragmented collection of tools for task tracking, equipment booking, scheduling, and internal communication. This fragmentation leads to coordination overhead, duplicated effort, and missed deadlines as team members context-switch between platforms that do not share data.

This project addresses that problem by building a unified, web-based collaboration dashboard that centralises the workflows most critical to cross-functional teams. The prototype consolidates task management, calendar scheduling, equipment booking, personnel tracking, notifications, and project coordination into a single platform accessible through a modern browser interface.

2 System Architecture

The application follows a clear client-server separation with a RESTful API boundary between frontend and backend. This decoupled architecture was chosen deliberately: it allows the frontend and backend to be developed, tested, and deployed independently, and ensures that either layer can be replaced or scaled without affecting the other – a practical consideration for a prototype that may evolve into a production system.

Backend is implemented in Rust using the Actix-web framework. Rust was selected for its compile-time memory safety guarantees and strong type system, which eliminate entire classes of bugs (null pointer dereferences, data races, buffer overflows) that are common in web backends written in dynamically typed or garbage-collected languages. Actix-web consistently ranks among the highest-performance web frameworks in industry benchmarks, making it well-suited for an API server handling concurrent requests from multiple dashboard users. Persistent data is stored in a SQLite database accessed via the `sqlx` async query library, which validates SQL queries against the database schema at compile time. The server exposes all endpoints under a common `/api` scope, organised into 13 route modules. The middleware stack handles CORS, JWT authentication enforcement, and request logging. A background task runs on an hourly schedule to clean up expired sessions.

Frontend is built with React and TypeScript. React was chosen for its mature component model and extensive ecosystem, while TypeScript adds static type checking that catches integration errors before runtime – particularly valuable when consuming a typed backend API. The component library is `shadcn/ui`, selected because it provides accessible, customisable primitives rather than opaque pre-built widgets, giving full control over styling and behaviour. Approximately 55 reusable UI components are composed into 13 primary page views. The frontend communicates exclusively with the backend via the REST API, with no shared in-process state between layers.

3 Key Features

3.1 Team Dashboard

The landing dashboard aggregates organisation-wide metrics in a single view: active project counts, total team members, and department breakdowns. This gives managers and leads immediate situational awareness without drilling into individual modules.

3.2 Task Management

Full CRUD task management with support for multiple assignees per task. Tasks carry urgency levels, status tracking, and a complete history log. Specialised views surface daily tasks and urgent items so team members can prioritise effectively.

3.3 Calendar and Event Scheduling

Users can create calendar events with attendee lists. An integrated calendar view provides a standard date-grid interface over all scheduled events, enabling teams to coordinate without resorting to external calendar tools.

3.4 Equipment Booking

An equipment inventory tracks assets by category and serial number. The booking system validates availability before confirming reservations and records maintenance dates to flag equipment nearing service intervals.

3.5 Personnel Tracking

A tracker-trackee relationship model allows authorised users to monitor team member locations and activity status. Personnel can follow specific colleagues, supporting field team coordination and on-site safety workflows.

3.6 Notifications

Multi-channel notification delivery covers email, browser push, and in-app alerts. A key design decision was implementing notification batching: web push notifications are accumulated within 3-second windows and dispatched as a single consolidated alert. This prevents notification overclustering – a common usability problem where rapid successive events (e.g. multiple task assignments or calendar updates) flood the user with individual alerts, causing notification fatigue and leading users to disable notifications entirely.

3.7 Additional Features

- **Glossary** – A searchable technical terminology reference for onboarding and cross-team alignment.
- **Global Search** – Cross-entity search spanning tasks, projects, equipment, and personnel records.
- **Quick Links** – Bookmarked URL management for frequently accessed internal resources.
- **Project Management** – Project lifecycle tracking with role-based membership distinguishing owners, admins, and members.

4 Implementation Details

4.1 Authentication

JWT tokens are issued on login and validated by middleware on every protected route. JWT was chosen over session-only authentication because it enables stateless request validation – the server can verify a token’s authenticity without a database lookup on every request, reducing latency for the frequent API calls a dashboard application generates. Passwords are hashed with bcrypt, an industry-standard adaptive hashing algorithm that remains resistant to brute-force attacks as hardware improves. Session records are stored in the database as a secondary layer and purged automatically by the hourly background cleanup task, limiting the accumulation of stale credentials.

4.2 API Routes

The 13 route modules registered under `/api` are: `auth`, `dashboard`, `users`, `calendar/events`, `tasks`, `equipment`, `locations`, `projects`, `quick_links`, `glossary`, `notifications`, `search`, and `tracking`. Each module encapsulates the handlers, request/response types, and database queries for its domain.

4.3 Database

SQLite is accessed through 12 model files that map to the core entities: users, equipment, tasks, events, projects, notifications, glossary entries, locations, quick links, tracking relationships, and sessions. SQLite was selected for its zero-administration deployment profile – it requires no separate database server process, stores the entire database in a single file, and supports the full SQL feature set needed by the application. This makes it ideal for a prototype where rapid iteration and simple deployment matter more than horizontal scalability. Migration to PostgreSQL or another managed database would be straightforward due to the use of standard SQL through `sqlx`.

4.4 Frontend Components

The 13 primary page components are: Login, Register, TeamDashboard, Dashboard-Calendar, TaskManagement, EquipmentBooking, LocationTracker, QuickLinks, Glossary, NotificationCenter, Personnel, Projects, and Profile, plus a UserTracking view. These map closely to the backend route modules, keeping the conceptual model consistent across layers.

4.5 Background Tasks

Two recurring background processes run independently of the request-handling threads: the hourly session cleanup, and the notification batching window manager, which accumulates push events over a configurable interval before dispatching them as a single consolidated notification.

5 Conclusion

The IT-Engineering Collaboration Dashboard prototype demonstrates the feasibility of consolidating fragmented team workflows into a single cohesive platform. The system successfully delivers task management, calendar scheduling, equipment booking, personnel tracking, notifications, and project coordination through a unified interface.

The Rust backend provides the type safety and runtime performance necessary to support concurrent API consumers, while the React and TypeScript frontend delivers a responsive, component-driven user experience. The clean API boundary between layers means either side can be extended or replaced independently as requirements evolve.

Future development would focus on production hardening – migrating to a managed relational database, adding role-based access control beyond the current project-level membership model, and conducting user testing with representative IT and Engineering teams to validate the workflow assumptions embedded in the current feature set.